

LAPORAN PRAKTIKUM PEKAN 2

PEMOGRAMAN BERORIENTASI OBJEK



Disusun oleh:

ANNAFI AL GHIFARI

2511533013

Dosen pengampu

Nurfiah, S.ST, M.Kom.

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2025

DAFTAR ISI

PENDAHULUAN	3
PROGRAM.....	4
KESIMPULAN.....	13

PENDAHULUAN

1.1 Latar Belakang

Konsep pemrograman berorientasi objek (Object-Oriented Programming/OOP) merupakan salah satu paradigma pemrograman yang banyak digunakan dalam pengembangan perangkat lunak modern. Salah satu penerapan OOP yang umum dijumpai adalah sistem simulasi rekening bank, yang melibatkan pengelolaan data nasabah, saldo, serta pencatatan riwayat transaksi. Melalui studi kasus ini, konsep class, object, enkapsulasi, serta penggunaan struktur data koleksi (ArrayList) dapat dipahami secara lebih konkret.

Pada praktikum ini dibuat sebuah program simulasi rekening bank sederhana berbasis konsol menggunakan bahasa pemrograman Java. Program terdiri atas tiga class utama, yaitu Main, Rekening, dan Transaksi, yang saling berinteraksi untuk menjalankan fungsi setor tunai, tarik tunai, pengecekan saldo, pergantian akun aktif, serta pencetakan mutasi dan informasi transaksi.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami penerapan konsep class dan object dalam bahasa pemrograman Java.
2. Memahami penerapan enkapsulasi melalui penggunaan modifier private serta method getter.
3. Memahami penggunaan struktur data ArrayList untuk menyimpan riwayat transaksi secara dinamis.
4. Memahami interaksi antar-objek (asosiasi/komposisi) antara class Rekening dan class Transaksi.
5. Mampu mengimplementasikan struktur kontrol perulangan (while) dan percabangan (switch-case) pada program berbasis menu.
6. Mampu menganalisis alur logika program simulasi rekening bank dari sisi antarmuka konsol hingga pengolahan data.

1.3 Rumusan Masalah

Manfaat dari praktikum ini adalah:

1. Bagaimana rancangan class yang tepat untuk merepresentasikan sebuah rekening bank beserta transaksinya?
2. Bagaimana cara memvalidasi transaksi setor dan tarik tunai agar sesuai dengan batas dan aturan yang ditetapkan?
3. Bagaimana cara menghitung saldo rekening secara akumulatif berdasarkan riwayat transaksi yang tersimpan?


```

        case 2:
            System.out.print("Masukkan nominal setor: ");
            double nominalSetor = scanner.nextDouble();
            if (akunAktif.setorTunai(nominalSetor)) {
                System.out.println("Setor tunai berhasil.");
            } else {
                System.out.println("Setor tunai gagal, Maksimal transaksi 500.000");
            }
            break;
        case 3:
            System.out.print("Masukkan nominal tarik: ");
            double nominalTarik = scanner.nextDouble();
            if (akunAktif.tarikTunai(nominalTarik)) {
                System.out.println("Penarikan berhasil.");
            } else {
                System.out.println("Penarikan gagal! Saldo tidak mencukupi atau melebihi batas
transaksi 500.000.");
            }
            break;
        case 4:
            System.out.println("\n--- INFORMASI REKENING ---");
            System.out.println("Nomor Rekening : " + akunAktif.getNomorRekening());
            System.out.println("Saldo          : " + akunAktif.getSaldo());
            break;
        case 5:
            if (akunAktif == akun1) {
                akunAktif = akun2;
            } else {
                akunAktif = akun1;
            }
            System.out.println("Berhasil berganti akun! Akun aktif saat ini: " +
akunAktif.getNomorRekening());
            break;
        case 6:
            akunAktif.cetakMutasi();
            break;
        case 7:
            akunAktif.cetakInformasi();
            break;
        case 0:
            berjalan = false;
            System.out.println("Terima kasih telah menggunakan layanan kami.");
            break;
        default:
            System.out.println("Pilihan tidak valid.");
    }
}
scanner.close();
}
}

```

2.2.2 Penjelasan

a. Membuat dua objek dari class yang sama

```

Rekening akun1 = new Rekening("123", 50000);
Rekening akun2 = new Rekening("456", 100000);

```

Baris ini menginstansiasi dua objek Rekening yang berbeda dari class yang sama. Masing-masing objek memiliki nomor rekening dan saldo awal sendiri, membuktikan bahwa satu class dapat menghasilkan banyak objek dengan data (state) yang terpisah.

b. Variabel penunjuk akun yang sedang aktif

Rekening akunAktif = akun1;

akunAktif adalah variabel referensi yang menunjuk ke salah satu objek Rekening. Karena hanya berisi referensi (bukan salinan data), mengganti nilai akunAktif cukup membuatnya menunjuk ke objek lain — inilah dasar dari fitur "Ganti Akun".

c. Perulangan menu utama

```
while (berjalan) {  
    ... tampilkan menu ...  
}
```

Perulangan while digunakan agar menu terus ditampilkan berulang kali. Perulangan ini baru berhenti ketika variabel boolean berjalan diubah menjadi false, yaitu saat pengguna memilih menu 0 (Keluar).

d. Percabangan berdasarkan pilihan menu

```
switch (pilihan) {  
    case 1: ... break;  
    case 2: ... break;  
    ...  
}
```

Struktur switch-case memetakan angka yang diinput pengguna ke aksi yang sesuai. Setiap case memanggil method milik akunAktif (misalnya akunAktif.getSaldo()), sehingga Main tidak perlu tahu detail perhitungan — cukup memanggil method yang sudah disediakan class Rekening.

e. Validasi hasil transaksi

```
if (akunAktif.setorTunai(nominalSetor)) {  
    System.out.println("Setor tunai berhasil.");  
} else {  
    System.out.println("Setor tunai gagal, Maksimal transaksi 500.000");  
}
```

Method setorTunai() mengembalikan nilai boolean (true/false). Main hanya perlu memeriksa hasilnya untuk menampilkan pesan berhasil atau gagal, sedangkan seluruh logika validasi nominal disimpan di dalam class Rekening.

f. Mekanisme ganti akun

```
if (akunAktif == akun1) {  
    akunAktif = akun2;  
} else {  
    akunAktif = akun1;  
}
```

Operator == di sini membandingkan referensi objek (apakah akunAktif sedang menunjuk ke objek yang persis sama dengan akun1), bukan membandingkan isi datanya. Jika ya, akunAktif dialihkan ke akun2, dan sebaliknya.

2.3 Program Rekening.java

2.3.1 Kode Program Rekening.java

```
package pekan2;  
  
import java.util.ArrayList;  
  
public class Rekening {  
    private String nomorRekening;  
    private double saldoAwal;  
    private ArrayList<Transaksi> riwayatTransaksi;  
  
    public Rekening(String nomorRekening, double saldoAwal) {  
        this.nomorRekening = nomorRekening;  
        this.saldoAwal = saldoAwal;  
        this.riwayatTransaksi = new ArrayList<>();  
    }  
  
    public boolean tarikTunai(double nominal) {  
        if (nominal > 0 && nominal <= 5000000 && nominal <= getSaldo()) {  
            String kodeTransaksi = "TRX-T-" + (riwayatTransaksi.size() + 1);  
            Transaksi trx = new Transaksi(kodeTransaksi, "Debit", nominal);  
            riwayatTransaksi.add(trx);  
            return true;  
        } else {  
            return false;  
        }  
    }  
}
```

```
public boolean setorTunai(double nominal) {
    if (nominal > 0 && nominal <= 5000000) {
        String kodeTransaksi = "TRX-S-" + (riwayatTransaksi.size() + 1);
        Transaksi trx = new Transaksi(kodeTransaksi, "Kredit", nominal);
        riwayatTransaksi.add(trx);
        return true;
    } else {
        return false;
    }
}

public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi trx : riwayatTransaksi) {
            trx.cetakDetail();
        }
    }
}

public void cetakInformasi() {
    double totalSetor = 0;
    double totalTarik = 0;

    for (Transaksi trx : riwayatTransaksi) {
        if (trx.getJenisTransaksi().equalsIgnoreCase("Kredit")) {
            totalSetor += trx.getNominal();
        } else if (trx.getJenisTransaksi().equalsIgnoreCase("Debit")) {
            totalTarik += trx.getNominal();
        }
    }

    double akumulasi = totalSetor - totalTarik;
    double saldoAkhir = saldoAwal + akumulasi;

    System.out.println("\n--- INFO TRANSAKSI ---");
    System.out.println("Total Setor : Rp " + totalSetor);
    System.out.println("Total Tarik : Rp " + totalTarik);
    System.out.println("Akumulasi : Rp " + akumulasi);
    System.out.println("Saldo : Rp " + saldoAkhir);
}
```

```

public double getSaldo() {
    double akumulasi = 0;
    for (Transaksi trx : riwayatTransaksi) {
        if (trx.getJenisTransaksi().equalsIgnoreCase("Kredit")) {
            akumulasi += trx.getNominal();
        } else if (trx.getJenisTransaksi().equalsIgnoreCase("Debit")) {
            akumulasi -= trx.getNominal();
        }
    }
    return saldoAwal + akumulasi;
}

public String getNomorRekening() {
    return nomorRekening;
}
}

```

2.3.2 Penjelasan

a. Atribut private (enkapsulasi)

```

private String nomorRekening;
private double saldoAwal;
private ArrayList<Transaksi> riwayatTransaksi;

```

Ketiga atribut dideklarasikan private sehingga tidak dapat diakses langsung dari class lain seperti Main. Data hanya bisa dibaca atau diubah melalui method publik yang disediakan (getSaldo(), setorTunai(), dan seterusnya) — inilah penerapan enkapsulasi.

b. Validasi dan pencatatan penarikan

```

if (nominal > 0 && nominal <= 500000 && nominal <= getSaldo()) {
    String kodeTransaksi = "TRX-T-" + (riwayatTransaksi.size() + 1);
    Transaksi trx = new Transaksi(kodeTransaksi, "Debit", nominal);
    riwayatTransaksi.add(trx);
    return true;
} else {
    return false;
}
}

```

Method tarikTunai() hanya meloloskan transaksi jika nominal positif, tidak melebihi Rp 500.000, dan tidak melebihi saldo saat ini. Jika lolos, dibuat objek Transaksi baru berjenis "Debit" dan disimpan ke riwayatTransaksi; saldo tidak diubah langsung, melainkan akan dihitung ulang dari riwayat ini.

c. Validasi dan pencatatan setoran

```
if (nominal > 0 && nominal <= 500000) {  
    String kodeTransaksi = "TRX-S-" + (riwayatTransaksi.size() + 1);  
    Transaksi trx = new Transaksi(kodeTransaksi, "Kredit", nominal);  
    riwayatTransaksi.add(trx);  
    return true;  
}
```

Method setorTunai() mirip dengan tarikTunai(), namun tidak perlu mengecek kecukupan saldo — hanya nominal positif dan tidak melebihi batas Rp 500.000. Transaksi yang tercatat berjenis "Kredit".

d. Perhitungan saldo secara akumulatif

```
double akumulasi = 0;  
for (Transaksi trx : riwayatTransaksi) {  
    if (trx.getJenisTransaksi().equalsIgnoreCase("Kredit")) {  
        akumulasi += trx.getNominal();  
    } else if (trx.getJenisTransaksi().equalsIgnoreCase("Debit")) {  
        akumulasi -= trx.getNominal();  
    }  
}  
return saldoAwal + akumulasi;
```

Program ini tidak menyimpan saldo sebagai satu variabel yang diubah-ubah, melainkan menghitungnya ulang setiap kali getSaldo() dipanggil, dengan menelusuri seluruh riwayatTransaksi: menjumlahkan semua Kredit dan mengurangi semua Debit, lalu ditambahkan ke saldoAwal. Pendekatan ini membuat saldo selalu konsisten dengan histori transaksi yang tercatat.

e. Mencetak riwayat transaksi

```
for (Transaksi trx : riwayatTransaksi) {  
    trx.cetakDetail();  
}  
cetakMutasi() menelusuri seluruh objek Transaksi dalam riwayatTransaksi dan memanggil method cetakDetail() milik masing-masing objek untuk menampilkan detailnya satu per satu.
```

2.4 Program Transaksi.java

2.4.1 Kode Program Transaksi.java

```
package pekan2;

public class Transaksi {
    String kodeTransaksi;
    String jenisTransaksi;
    double nominal;

    public Transaksi(String kodeTransaksi, String jenisTransaksi, double nominal) {
        this.kodeTransaksi = kodeTransaksi;
        this.jenisTransaksi = jenisTransaksi;
        this.nominal = nominal;
    }

    public String getKodeTransaksi() {
        return kodeTransaksi;
    }

    public String getJenisTransaksi() {
        return jenisTransaksi;
    }

    public double getNominal() {
        return nominal;
    }

    public void cetakDetail() {
        System.out.println(kodeTransaksi + " | " + jenisTransaksi + " | Rp " +
            nominal);
    }
}
```

2.4.2 Penjelasan

a. Atribut penyusun satu transaksi

String kodeTransaksi;

String jenisTransaksi;

double nominal;

Satu objek Transaksi hanya merepresentasikan satu baris transaksi: kode unik, jenis ("Kredit" atau "Debit"), dan nominalnya. Karena disimpan di dalam ArrayList milik Rekening, satu objek Rekening dapat memiliki banyak objek Transaksi (hubungan one-to-many).

b. Mencetak detail satu transaksi

```
public void cetakDetail() {  
    System.out.println(kodeTransaksi + " | " + jenisTransaksi + " | Rp " +  
nominal);  
}
```

Method ini dipanggil berulang kali oleh Rekening.cetakMutasi() untuk mencetak setiap transaksi dalam format kode | jenis | nominal, sehingga riwayat transaksi mudah dibaca.

KESIMPULAN

1. Program simulasi rekening bank ini berhasil mengimplementasikan konsep class, object, dan enkapsulasi melalui pemisahan tanggung jawab antara class Main, Rekening, dan Transaksi.
2. Enkapsulasi diterapkan dengan mendeklarasikan atribut sebagai private dan menyediakan method publik (getter dan method aksi) sebagai satu-satunya cara mengakses maupun memodifikasi data rekening.
3. Struktur data ArrayList<Transaksi> terbukti efektif digunakan untuk menyimpan riwayat transaksi secara dinamis tanpa perlu menentukan ukuran koleksi di awal.
4. Validasi nominal transaksi (batas maksimal Rp 500.000 dan kecukupan saldo) berhasil mencegah terjadinya transaksi yang tidak valid.
5. Program ini dapat dikembangkan lebih lanjut dengan penambahan validasi input, pemformatan mata uang, serta fitur transfer antar-rekening untuk meningkatkan kegunaannya.